

UNIVERSIDAD ANDINA DEL CUSCO
FACULTAD DE INGENIERÍA Y ARQUITECTURA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



SPRINT 3

EQUIPO: Fuerza Popular

ASIGNATURA: Inteligencia Artificial

DOCENTE: Mg. Ing. Hugo Espetia Huamanga

INTEGRANTES:

- Huarcaya Cabrera Yajaira ... 100%
- La Torre Velazco Anthony Jareth ... 100%
- Socualaya Olivera, Dante Joel (SM) ... 100%
- Tresierra Zamora Diego Andres ... 100%

Cusco-Perú

2026 - I

Introducción

El presente documento detalla el cierre y los resultados obtenidos durante el Sprint 3 del proyecto Hackermate. Este proyecto tiene como visión a largo plazo el desarrollo de un juego de ajedrez interactivo que enfrente a un jugador humano contra un modelo de Inteligencia Artificial.

Durante las fases previas (Sprints 1 y 2), se estableció la arquitectura base del software, la interfaz gráfica de usuario (GUI) utilizando Python y la validación de movimientos geométricos básicos. El objetivo crítico de este tercer sprint ha sido la transformación de ese tablero estático en un motor de ajedrez completo y riguroso, capaz de arbitrar una partida de "Jugador vs Jugador" (PvP) aplicando las reglas avanzadas del ajedrez convencional, garantizando un entorno seguro y preparado para la futura integración del modelo de IA.

Información General

- **Proyecto:** Hackermate (Motor de Ajedrez + Interfaz PvP)
- **Fase / Sprint:** Sprint 3
- **Fecha de Cierre:** 26 de abril de 2026
- **Rama de Integración:** main (Merge desde feature/sprint3-ajedrez)
- **Responsable Técnico:** Dante

Objetivo del Sprint

Completar y asegurar el motor lógico del juego. El objetivo principal fue evolucionar el tablero de una simple matriz de movimientos a un motor de ajedrez restrictivo, capaz de identificar amenazas (Jaque), evitar movimientos ilegales, procesar reglas especiales (Promoción, Enroque) y determinar el fin de la partida de forma autónoma.

ÉPICA: Motor de Reglas Avanzadas y Arbitraje Autónomo Descripción: Dotar al sistema de la capacidad de arbitrar una partida de ajedrez completa, validando no solo la geometría de los movimientos, sino el estado global del tablero, las restricciones de seguridad del Rey y las condiciones de fin de juego, preparando el entorno para la futura interacción con la IA.

Historias de Usuario (Sprint Backlog)

ID	Título	Historia de Usuario (Como / Quiero / Para)	Criterios de Aceptación
HU-301	Promoción del Peón	Como jugador, quiero que mi peón se convierta automáticamente en una reina al llegar a la última fila enemiga, para obtener una ventaja estratégica.	1. Si un peón blanco llega a la fila 0, su tipo cambia a "reina". 2. Si un peón oscuro llega a la fila 7, su tipo cambia a "reina". 3. La interfaz actualiza el sprite inmediatamente.
HU-302	Detección de Jaque	Como jugador, quiero ser notificado visualmente cuando mi rey esté siendo amenazado, para poder tomar medidas defensivas.	1. El motor escanea si alguna pieza enemiga tiene un movimiento válido hacia el rey actual. 2. El panel lateral cambia a color rojo.

			3. Se muestra el texto "¡ESTÁS EN JAQUE!".
HU-303	Bloqueo de Movimientos Ilegales	Como motor de juego, quiero evitar que un jugador realice un movimiento que deje a su propio rey en jaque, para garantizar el cumplimiento estricto de las reglas.	1. El sistema simula internamente el movimiento antes de confirmarlo. 2. Si la simulación resulta en auto-jaque, el clic se ignora (retorna False). 3. El turno no cambia hasta que se realice un movimiento legal.
HU-304	Condición de Jaque Mate	Como jugador, quiero que el juego detecte automáticamente cuando no hay movimientos salvavidas disponibles, para que la partida finalice y se declare un ganador.	1. El motor simula todos los movimientos posibles del jugador en turno. 2. Si ningún movimiento elimina el estado de jaque, se declara "Jaque Mate". 3. La interfaz bloquea nuevos clics en el tablero. 4. El panel lateral se vuelve dorado y anuncia al ganador.
HU-305	Enroque (Corto y Largo)	Como jugador, quiero poder mover mi rey dos casillas hacia una torre y que la torre salte al otro lado, para proteger a mi rey rápidamente.	1. Se valida que ni el rey ni la torre involucrada se hayan movido antes (ha_movido = False). 2. Se valida que no haya piezas intermedias. 3. Se valida que el rey no esté en jaque, ni pase por una casilla en jaque.

HU-306	Condición de Rey Ahogado	Como jugador, quiero que el juego detecte si me quedo sin movimientos legales sin estar en jaque, para que la partida se declare en empate (Tablas).	1. El jugador no tiene movimientos válidos en todo el tablero. 2. El rey NO está actualmente amenazado. 3. El panel lateral muestra el texto "¡TABLAS!" en color gris.
---------------	--------------------------	---	--

Resumen de Entregables (Features)

A continuación, se detallan las características implementadas durante el ciclo de desarrollo:

ID	Característica	Descripción Lógica y Visual	Archivos Afectados	Estado
F-01	Promoción del Peón	Mutación automática del objeto Pieza de tipo "peon" a "reina" al alcanzar la fila 0 (blancas) o fila 7 (oscuras). Actualización gráfica en tiempo real.	tablero.py	OK
F-02	Detección de Jaque	Algoritmo de escaneo de matriz que verifica si la posición actual del Rey está bajo la línea de ataque de cualquier pieza enemiga.	reglas.py	OK
F-03	Bloqueo de Movimientos Ilegales	Sistema de simulación de estado temporal. Previene que un jugador realice un movimiento que deje o ponga a su propio Rey en Jaque.	tablero.py	OK

F-04	Jaque Mate y Ahogado	Evaluación predictiva de la matriz. Si un jugador no tiene movimientos válidos, el sistema evalúa si está en Jaque (Jaque Mate) o no (Tablas) y bloquea la interfaz.	tablero.py, juego_pvp.py	OK
F-05	Enroque (Corto y Largo)	Validación de regla compleja. Evalúa el estado histórico de las piezas (ha_movido), espacios vacíos y verifica que el Rey no transite por casillas bajo amenaza.	piezas.py, reglas.py, tablero.py	OK
F-06	UI Reactiva	Modificación del panel lateral para mostrar alertas de estado ("¡ESTÁS EN JAQUE!", "¡JAQUE MATE!", "¡TABLAS!") mediante cambios de color dinámicos.	juego_pvp.py	OK

Detalles de Arquitectura y Decisiones Técnicas

Durante este sprint, la arquitectura del software requirió los siguientes ajustes:

- **Inyección de Estado Histórico:** Se modificó la clase base Pieza (piezas.py) introduciendo la propiedad booleana `ha_movido`. Esto permite al motor mantener un registro del estado inicial sin necesidad de almacenar todo el historial de la partida, optimizando el consumo de memoria para la validación del Enroque.
- **Patrón de Simulación de Matriz (Rollback):** Para la validación del "Auto-Jaque" y el "Jaque Mate", se implementó un algoritmo predictivo de fuerza bruta superficial (profundidad 1) en `tablero.py`.
- **Desacoplamiento de UI y Lógica:** La interfaz gráfica (`juego_pvp.py`) se mantiene agnóstica de las reglas matemáticas. La UI solo evalúa si `tablero.mover_pieza()` retorna `True` o `False` para cambiar de turno o ignorar el clic, respetando el patrón MVC (Modelo-Vista-Controlador).

Deuda Técnica Y Consideraciones

- **Captura al paso:** Actualmente el motor no soporta esta regla específica del peón. Se registra como deuda técnica por si se requiere su implementación en el futuro.
- **Optimización:** La función tiene `_movimientos_validos` revisa todo el tablero en cada turno. Para partidas PvP es imperceptible, pero cuando se integre la IA, esta función deberá optimizarse.

Preparación Sprint 4

Con el motor del juego estrictamente reglado y funcional, el proyecto se encuentra listo para iniciar la investigación y desarrollo de la Modalidad IA.

- Iniciar el diseño del algoritmo de búsqueda o preparar el entorno para la recolección de datos y entrenamiento del modelo de red neuronal.
- Desarrollar la vista `juego_ia.py` para que el sistema asuma el rol de Jugador 2.

Conclusión

El Sprint 3 concluye con éxito, entregando un producto de software robusto, modular y 100% funcional en su modalidad de Jugador vs Jugador. Se ha superado el desafío de programar las validaciones lógicas más complejas del ajedrez sin comprometer la fluidez de la interfaz gráfica ni la limpieza del código.

Al establecer un motor de reglas estricto que gestiona autónomamente los estados de victoria, amenaza y restricciones, se ha creado la base fundamental e indispensable para el siguiente hito del proyecto. A partir del Sprint 4, el equipo se enfocará exclusivamente en la investigación, estructuración y entrenamiento del modelo de Inteligencia Artificial, el cual interactuará directamente con este motor consolidado a través de la interfaz `juego_ia.py`.

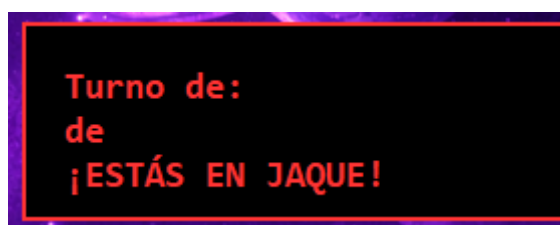
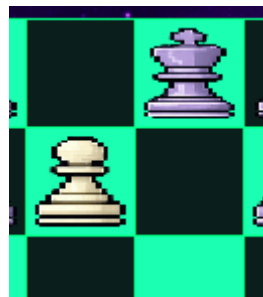
ANEXOS

- Promoción del Peón

```
PS D:\Dante UAC\Hackermate\hackermate> python main.py
PS D:\Dante UAC\Hackermate\hackermate> git add game/tablero.py
PS D:\Dante UAC\Hackermate\hackermate> git commit -m "feat: implementar promocion automatica del peon a reina"
[feature/sprint3-ajedrez 91bb3ff] feat: implementar promocion automatica del peon a reina
1 file changed, 8 insertions(+)
```



- Detección de Jaque

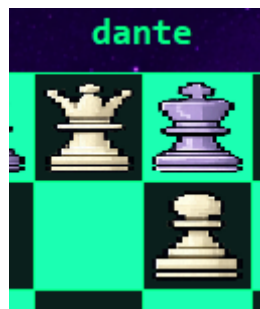


```
PS D:\Dante UAC\Hackermate\hackermate> git add screen/juego_pvp.py
PS D:\Dante UAC\Hackermate\hackermate> git commit -m "ui: agregar indicador visual de JAQUE en el panel de turnos"
[feature/sprint3-ajedrez 1fdf926] ui: agregar indicador visual de JAQUE en el panel de turnos
1 file changed, 16 insertions(+), 2 deletions(-)
```

- Bloqueo de Movimientos Ilegales

```
PS D:\Dante UAC\Hackermate\hackermate> git add game/tablero.py
PS D:\Dante UAC\Hackermate\hackermate> git commit -m "fix: simular movimientos para bloquear auto-jaque y movimientos ilegales"
[feature/sprint3-ajedrez bca155b] fix: simular movimientos para bloquear auto-jaque y movimientos ilegales
1 file changed, 25 insertions(+), 8 deletions(-)
```

- Jaque Mate y Ahogado



¡JAQUE MATE!
Ganó el rival

- Enroque (Corto y Largo)

